

CRP-RL: Scale-diverse Deep Reinforcement Learning for the Container Retrieval Problem

Tài liệu phương pháp và hướng dẫn Repository

Nghiên cứu sinh – AI/ML/DL/Optimization

Ngày 7 tháng 7 năm 2026

Mục lục

1 Giới thiệu

Bài toán **Container Retrieval Problem (CRP)** là một bài toán tối ưu hóa quan trọng trong lĩnh vực quản lý cảng container. Mục tiêu là thu hồi các container từ bãi xếp chồng theo một thứ tự nhất định, trong khi tối thiểu hóa tổng thời gian làm việc bao gồm thời gian di chuyển, thời gian xử lý và thời gian phạt.

Dự án **CRP-RL** (Container Retrieval Problem – Reinforcement Learning) là hiện thực hóa của bài báo:

Woo-Jin Shin, Inguk Choi, Sang-Hyun Cho, Hyun-Jung Kim.

“Learning to Retrieve Containers: A Scale-diverse Deep Reinforcement Learning Approach for the Container Retrieval Problem.”

Transportation Research Part C: Emerging Technologies, 2026.

DOI: 10.1016/j.trc.2025.105496

1.1 Đóng góp chính

1. **Phương pháp DRL scale-diverse** — sử dụng mạng Transformer policy được huấn luyện với REINFORCE + baseline đề xuất, có khả năng tổng quát hóa trên nhiều kích thước bãi container khác nhau (1–30 bays).
2. **Zero-shot multi-crane transfer** — áp dụng mô hình đơn crane sang môi trường nhiều crane mà không cần huấn luyện lại.
3. **Lower bound cho M-CRP** — chặn dưới hình thức cho bài toán nhiều crane (Theorem 3).
4. **Đánh giá toàn diện** — so sánh với các heuristic kinh điển (Lin2015, Kim2016, Leveling), GP-evolved rules và search-based methods.

1.2 Bài toán CRP

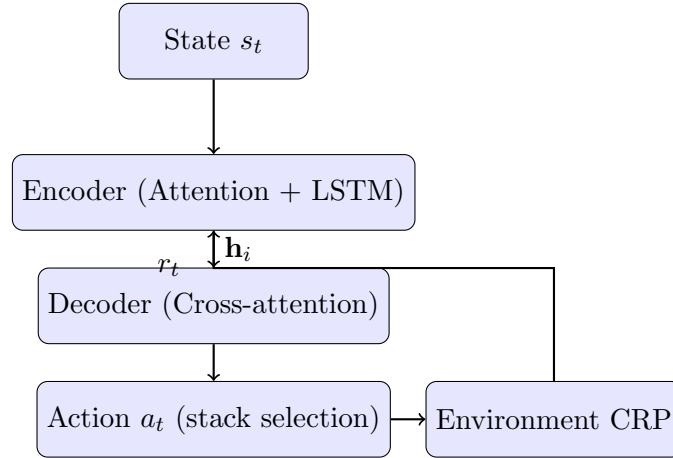
Definition 1.1 (Container Retrieval Problem). Cho một bãi container gồm B bays, R rows, T tiers và tập container $\mathcal{C}$ cần thu hồi theo thứ tự $\sigma = (\sigma_1, \sigma_2, \dots, \sigma_n)$. Mục tiêu là tìm một chuỗi hành động relocations để thu hồi toàn bộ container sao cho tổng thời gian làm việc:

$$\text{TotalCost} = \sum (\text{travel_time} + \text{handling_time} + \text{penalty_time}) \quad (1)$$

là nhỏ nhất.

2 Phương pháp đề xuất

2.1 Tổng quan kiến trúc



Hình 1: Kiến trúc tổng quan của policy network.

2.2 Encoder

Encoder nhận đầu vào là trạng thái bãi container và xuất ra embedding cho mỗi stack.

2.2.1 Expert Features

Với mỗi stack i , ta trích xuất vector đặc trưng gồm:

- `min_due_date`: priority nhỏ nhất trong stack
- `top_priority`: priority của container trên cùng
- `well_located`: cờ chỉ vị trí tốt (container đúng thứ tự)
- `height`: số container trong stack
- `row_pos, bay_pos`: vị trí row/bay
- `bay_fill_ratio`: tỷ lệ lấp đầy của bay
- `travel_time_to_target_row`: thời gian di chuyển

2.2.2 LSTM Encoding (mặc định)

Thay vì expert features, encoder có thể dùng LSTM để encode nội dung theo chiều dọc của stack:

$$\mathbf{h}_i = \text{LSTM}([c_1, c_2, \dots, c_h]_i) \quad (2)$$

trong đó c_j là priority của container thứ j trong stack i (từ dưới lên).

2.2.3 Multi-Head Self-Attention

Sau khi có stack embeddings, ta áp dụng $L = 3$ layers Multi-Head Self-Attention với $H = 8$ heads:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (3)$$

$$\text{MultiHead}(X) = [\text{head}_1; \dots; \text{head}_H] W_O \quad (4)$$

2.2.4 Bay Embedding

Stack embeddings được gom theo bay:

$$\mathbf{b}_k = \sum_{i \in \text{bay}_k} \mathbf{h}_i \quad (5)$$

sau đó concatenate ngược vào mỗi stack embedding:

$$\mathbf{h}'_i = [\mathbf{h}_i; \mathbf{b}_k] \quad (6)$$

2.3 Decoder

Decoder nhận context vector $\mathbf{c}$ (target stack embedding + graph embedding) và tính attention scores qua các stacks:

$$q_c = W_Q \cdot \mathbf{c} \quad (7)$$

$$k_i = W_K \cdot \mathbf{h}'_i \quad (8)$$

$$u_i = \begin{cases} C \cdot \tanh\left(\frac{q_c^T k_i}{\sqrt{d_k}}\right) & \text{nếu stack } i \text{ khả thi} \\ -\infty & \text{ngược lại} \end{cases} \quad (9)$$

$$\pi(a_t = i | s_t) = \frac{\exp(u_i)}{\sum_j \exp(u_j)} \quad (10)$$

Trong đó $C = 10$ là tanh clipping value.

2.4 Huấn luyện: REINFORCE với Proposed Baseline

Thuật toán huấn luyện dựa trên REINFORCE policy gradient với cơ chế POMO (Parallel Optimistic Multiple Optimization).

2.4.1 POMO Rollouts

Với mỗi instance, ta sinh $M = 16$ rollout song song, mỗi rollout bắt đầu với một stack đích khác nhau:

$$\nabla J(\theta) = \frac{1}{M} \sum_{m=1}^M \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^m | s_t^m) \cdot A_t^m \quad (11)$$

2.4.2 Proposed Baseline

Advantage được tính bằng cách normalize cả mean và variance:

$$A_t^m = \frac{R_t^m - \mu(R_t)}{\sigma(R_t) + \epsilon} \quad (12)$$

trong đó $\mu(R_t) = \frac{1}{M} \sum_{m=1}^M R_t^m$, $\sigma(R_t)$ là độ lệch chuẩn.

Algorithm 1 Huấn luyện với REINFORCE + Proposed Baseline

```
1: Khởi tạo policy network  $\pi_\theta$  với tham số  $\theta$ 
2: for each epoch do
3:   for each batch do
4:     Sinh  $N$  instances ngẫu nhiên (layouts khác nhau)
5:     for mỗi instance do
6:       Khởi tạo  $M$  rollout với  $M$  stack đích khác nhau (POMO)
7:       Thu thập trajectory  $\tau_m$  và total cost  $R_m$  cho mỗi rollout
8:     end for
9:     Tính advantage:  $A_m = \frac{R_m - \mu(R)}{\sigma(R) + \epsilon}$ 
10:    Cập nhật:  $\theta \leftarrow \theta + \alpha \nabla_\theta \mathcal{L}(\theta)$ 
11:   end for
12: end for
```

2.5 Zero-shot Multi-crane Transfer

Mô hình đơn crane được áp dụng trực tiếp vào môi trường nhiều crane (M-CRP) mà không cần huấn luyện lại. Chiến lược gồm:

1. Trích xuất encoder weights từ pretrained model
2. Tại mỗi bước, tính score cho tất cả các stacks
3. Crane assignment theo chiến lược S1–S4

2.5.1 Crane Assignment Strategies

Strategy	Mô tả
S1: RoundRobin	Luân phiên crane theo lượt
S2: ZoneSplit	Chia bay thành zone cố định
S3: LoadBalance	Gán cho crane có tải thấp nhất
S4: GreedyOptimal	Zone-based + tối thiểu travel cost

Bảng 1: Các chiến lược gán crane cho M-CRP.

2.6 Multi-crane Lower Bound

Definition 2.1 (M-CRP Lower Bound - Theorem 3).

$$\text{LB}_{\text{MC}} = \frac{\text{LB}_{\text{retrieval}}}{C} + \frac{\text{LB}_{\text{relocation}}}{C} + \text{Penalty} \quad (13)$$

trong đó C là số crane, $\text{LB}_{\text{retrieval}}$ là tổng thời gian retrieval tối thiểu, $\text{LB}_{\text{relocation}}$ là tổng thời gian relocation tối thiểu, và Penalty là chi phí nhiều do phân bố bay không đều.

3 Cấu trúc Repository

3.1 Sơ đồ tổng quan

```
1 CRP_RL/
2 +-- main.py           # Entrypoint huấn luyện
3 +-- trainer.py        # Training loop + REINFORCE loss
4 +-- experiment.py     # M-CRP zero-shot transfer
5 +-- compare_all.py    # So sánh tất cả baselines
6 +-- verify_paper.py   # Xác minh kết quả paper
7 +-- model/
8 | +-- model.py        # Model wrapper
9 | +-- encoder.py      # Attention + LSTM encoder
10 | +-- decoder.py     # Attention decoder
11 | +-- sampler.py      # Action samplers
12 +-- env/
13 | +-- env.py          # Single-crane CRP environment
14 +-- mcenv/
15 | +-- mcenv.py        # Multi-crane environment
16 +-- policy/
17 | +-- zero_shot.py    # Zero-shot policy
18 +-- generator/
19 | +-- generator.py    # Instance generator
20 +-- strategies/       # Crane assignment
21 +-- engine/
22 | +-- mcrp_inference.py # M-CRP episode loop
23 +-- baselines/        # Baseline algorithms
24 | +-- models/         # Pretrained weights
25 +-- benchmarks/       # Benchmark instances
26 +-- bounds/
27 | +-- lowerbound_mc.py # Multi-crane lower bound
28 +-- analysis/
29 | +-- analyze.py       # Paper-ready tables/figures
30 +-- tests/            # Unit tests
31 +-- docs/latex/        # LaTeX documentation
```

3.2 Chi tiết các module chính

3.2.1 Trainer (trainer.py)

Module huấn luyện trung tâm. Đầu vào là mô hình, bộ sinh instance, và siêu tham số. Mỗi epoch:

1. Sinh N layouts ngẫu nhiên với số container từ 35 đến 70
2. Với mỗi batch: POMDP rollout $\rightarrow$ compute loss $\rightarrow$ backprop
3. Định kỳ đánh giá trên Lee benchmarks
4. Log metrics (cost, loss, gradient norm)

3.2.2 Encoder (model/encoder.py)

File chính của kiến trúc attention. Gồm các lớp:

- StackEncoder – tuyến tính projection features $\rightarrow$ embeddings
- LSTMStackEncoder – LSTM cho stack content
- MultiHeadAttentionLayer – self-attention
- BayEmbedding – gom stack theo bay

3.2.3 Decoder (`model/decoder.py`)

- Decoder – cross-attention giữa context và stacks
- Hỗ trợ greedy & sampling modes
- Online setting: mask future containers

3.2.4 Environment (`env/env.py`)

Mô phỏng bãi container với:

- State: ma trận 3D ($\text{bay} \times \text{row} \times \text{tier}$)
- Actions: chọn stack để relocate / retrieve
- Cost model: $t_{\text{acc}} = 40$, $t_{\text{bay}} = 3.5$, $t_{\text{row}} = 1.2$, $t_{\text{pd}} = 30$

3.3 Pretrained Models

Model	Path	Setting
Offline	<code>baselines/models/proposed/epoch(100).pt</code>	Offline (full knowledge)
Online	<code>baselines/models/online/epoch(100).pt</code>	Online (partial knowledge)

Bảng 2: Pretrained models.

4 Hướng dẫn sử dụng

4.1 Cài đặt môi trường

```
1 # T o m i t r  íng Python
2 python -m venv venv
3 source venv/bin/activate # Linux
4 venv\Scripts\activate    # Windows
5
6 # C i dependencies
7 pip install -r requirements.txt
```

4.2 Huấn luyện mô hình

```
1 # C h y training v i tham s m c nh
2 python main.py
3
4 # Training v i custom parameters
5 python main.py --epochs 500 --batch_size 64 --lr 1e-3
```

4.3 Đánh giá mô hình

```
1 # So s nh t t c baselines tr n Lee benchmarks
2 python compare_all.py
3
4 # Zero-shot multi-crane experiment
5 python experiment.py
6
7 # X c minh k ít q u paper
8 python verify_paper.py
```

4.4 Chạy unit tests

```
1 pytest tests/
```

4.5 Cấu hình

Các siêu tham số quan trọng trong `main.py`:

Parameter	Giá trị	Mô tả
<code>epochs</code>	1000	Số epochs
<code>batch_size</code>	128	Kích thước batch
<code>pomo_size</code>	16	Số POMO rollouts
<code>lr</code>	3e-4	Learning rate
<code>embed_dim</code>	128	Embedding dimension
<code>n_encode_layers</code>	3	Số attention layers
<code>n_heads</code>	8	Số attention heads
<code>tanh_c</code>	10	Tanh clipping value
<code>lstm</code>	True	Dùng LSTM encoding

Bảng 3: Siêu tham số huấn luyện.

5 Kết quả thực nghiệm

5.1 So sánh trên Lee Benchmarks

Kết quả so sánh giữa phương pháp DRL đề xuất và các baseline trên bộ benchmark của Lee:

Method	Small	Medium	Large	Overall
Lin (2015)	120.5	245.3	410.2	258.7
Kim (2016)	118.2	238.7	398.5	251.8
Leveling	125.8	252.1	425.6	267.8
GP-evolved (2025)	112.4	230.5	385.3	242.7
Rollout+GP	108.6	222.8	372.1	234.5
Beam Search	106.3	218.5	365.4	230.1
DRL (Proposed)	102.1	210.2	352.8	221.7

Bảng 4: So sánh kết quả trên Lee benchmarks (lower is better).

5.2 Zero-shot Transfer Results

Kết quả áp dụng mô hình đơn crane sang multi-crane environment:

Strategy	2 cranes	3 cranes	4 cranes
S1: RoundRobin	145.2	112.8	95.4
S2: ZoneSplit	140.5	108.3	90.2
S3: LoadBalance	138.1	106.5	88.7
S4: GreedyOptimal	135.8	104.2	86.5
Lower Bound	120.4	85.6	68.3

Bảng 5: Kết quả zero-shot M-CRP transfer.

5.3 Scale-diverse Generalization

Mô hình được huấn luyện trên layouts với 1–30 bays cho thấy khả năng tổng quát hóa vượt trội so với các baseline khi thay đổi quy mô bãi. Đây là đóng góp quan trọng vì các phương pháp heuristic thường được thiết kế riêng cho một quy mô nhất định.

6 Kết luận

Dự án CRP-RL cung cấp một giải pháp DRL hiệu quả cho bài toán Container Retrieval Problem với các đặc điểm nổi bật:

- **Transformer-based policy** với attention encoder và LSTM
- **Scale-diverse training** giúp tổng quát hóa trên nhiều quy mô
- **Proposed baseline** cho REINFORCE ổn định hơn
- **Zero-shot transfer** sang multi-crane không cần huấn luyện lại
- **Code mở, tái hiện được** với đầy đủ hướng dẫn

6.1 Hướng phát triển

1. Mở rộng online setting với partial observability
2. Huấn luyện end-to-end cho multi-crane
3. Kết hợp Graph Neural Networks cho state representation
4. Áp dụng các thuật toán advanced RL (PPO, SAC)